SMART HOME AUTOMATION AND SECURITY SYSTEM USING ARDUINO

Link:- https://www.tinkercad.com/things/4qfTaz2Iddu-smart-home-automation-system?sharecode=dzI8vQPfdNjFV1sPY2cMsdLhH6p7NaWS-NNuRb8MCzM

1. INTRODUCTION

Smart home systems are designed to make homes more convenient, energy-efficient, and secure by automatically responding to environmental conditions.

Traditional electrical appliances often require manual operation. For example, lights need to be switched ON manually when it becomes dark, fans need to be controlled according to temperature, and security systems require separate motion-detection equipment.

The proposed system integrates these functions into one Arduino-based platform.

The major functions implemented in this project are:

1. Automatic light control using an LDR.
2. Temperature monitoring using a TMP36 sensor.
3. Automatic fan control according to temperature.
4. Motion detection using a PIR sensor.
5. Security alert using an LED and buzzer.
6. LCD display for system status and temperature.
7. Monitoring of an additional analog sensor through A2.
8. Serial Monitor output for debugging and monitoring.

2. OBJECTIVES

The main objectives of this project are:

- To develop a basic smart-home automation system using Arduino.
- To automatically control lighting according to surrounding brightness.
- To measure temperature using a TMP36 temperature sensor.
- To automatically control a fan/motor when temperature exceeds a predefined limit.
- To detect human movement using a PIR sensor.
- To generate an audible and visual security alert when motion is detected.
- To display important information on a 16×2 LCD.
- To demonstrate the use of analog and digital sensors with Arduino.
- To monitor sensor values through the Serial Monitor.

3. COMPONENTS REQUIRED

1. Arduino Uno
2. 16×2 LCD Display
3. PIR Motion Sensor
4. TMP36 Temperature Sensor
5. LDR

6. Buzzer
7. 2 LEDs
8. Extra Analog Sensor
9. DC Motor/Fan
10. 220 Ω Resistors – 2
11. 10 kΩ Resistor – 1
12. NPN Transistor/MOSFET – 1
13. 1N4007 Diode – 1
14. Breadboard
15. Jumper Wires
16. USB Cable
17. External Power Supply

4. HARDWARE DESCRIPTION

4.1 Arduino Uno

Arduino Uno is the central controller of the project. It reads sensor values, processes the data, controls LEDs, buzzer, and motor, and sends information to the LCD and Serial Monitor.

4.2 PIR Motion Sensor

The PIR sensor is connected to digital pin 9.

When motion is detected:

- The PIR output becomes HIGH.
- The motion LED connected to pin 8 turns ON.
- The buzzer produces an alert tone.
- The LCD displays an alert message.

When no motion is detected, the motion LED is switched OFF and the LCD displays the normal system status.

4.3 LDR Sensor

The LDR is connected to analog input A0.

The Arduino reads the LDR value using:

```
analogRead(A0)
```

The program uses a threshold value of 500.

If:

LDR value < 500

the system assumes that the environment is dark and turns the light LED ON.

If:

LDR value ≥ 500

the system assumes that the environment is bright and turns the light LED OFF.

4.4 TMP36 Temperature Sensor

The TMP36 temperature sensor is connected to analog input A1.

The Arduino first converts the analog reading into voltage:

```
voltage = analog reading × (5.0 / 1023.0)
```

The TMP36 temperature is then calculated using:

```
Temperature (°C) = (Voltage - 0.5) × 100
```

The program uses 25°C as the fan activation threshold.

If:

Temperature > 25°C

the fan/motor is switched ON at full PWM power.

If:

Temperature ≤ 25°C

the fan/motor is switched OFF.

4.5 DC Motor/Fan

The motor is connected to Arduino pin 6.

Pin 6 supports PWM on the Arduino Uno, allowing the program to control motor power using `analogWrite()`.

The current program uses:

- `analogWrite(MOTOR_PIN, 255)` → Fan ON
- `analogWrite(MOTOR_PIN, 0)` → Fan OFF

Important: A DC motor should not be powered directly from an Arduino I/O pin. A transistor/MOSFET or motor-driver circuit and a suitable external power supply should be used.

4.6 16×2 LCD

The LCD is connected using the following pins:

LCD Function	Arduino Pin
RS	12
EN	11
D4	5
D5	4
D6	3
D7	2

The LCD displays system information such as:

- Smart System
- Starting...
- ALERT!!!
- Motion Detected
- System Normal
- No Motion
- Temperature
- Fan status

5. PIN CONFIGURATION

The program defines the following pins:

Device	Arduino Pin	Type
PIR Sensor	D9	Digital Input
Motion LED	D8	Digital Output
Buzzer	D7	Digital Output
Light LED	D13	Digital Output
LDR	A0	Analog Input
TMP36	A1	Analog Input
Extra Sensor	A2	Analog Input
Extra Output	D7	Digital Output
Motor/Fan	D6	PWM Output

LCD RS	D12	Output
LCD EN	D11	Output
LCD D4	D5	Output
LCD D5	D4	Output
LCD D6	D3	Output
LCD D7	D2	Output

Important Pin Conflict

There is a conflict in the supplied program:

```
const int BUZZER_PIN = 7;
const int EXTRA_OUTPUT = 7;
```

Both the buzzer and extra output are assigned to **Arduino pin 7**.

Therefore, the project should be corrected by assigning the extra output to another unused digital pin. For example:

```
const int EXTRA_OUTPUT = 10;
```

provided that pin 10 is not required for another device.

6. SOFTWARE USED

The project can be developed using:

- Arduino IDE
- Embedded C/C++ programming
- Arduino `LiquidCrystal` library
- Serial Monitor at 9600 baud

The LCD is initialized using:

```
lcd.begin(16, 2);
```

Serial communication is initialized using:

```
Serial.begin(9600);
```

7. WORKING PRINCIPLE

The system continuously executes the instructions inside the Arduino `loop()` function.

Step 1: Read Additional Sensor

The analog value from A2 is read.

If the value is below 500, the extra output is turned OFF.

If the value is 500 or higher, the extra output is turned ON.

Step 2: Measure Temperature

The Arduino reads the TMP36 sensor from A1.

The analog value is converted into voltage and then into temperature in degrees Celsius.

The temperature is displayed through the Serial Monitor and later on the LCD.

Step 3: Control Fan

The measured temperature is compared with 25°C.

Temperature > 25°C:

- Fan ON
- PWM = 255
- Serial Monitor reports Fan ON

Temperature ≤ 25°C:

- Fan OFF
- PWM = 0
- Serial Monitor reports Fan OFF

Step 4: Automatic Lighting

The LDR value is read from A0.

When the measured value is below 500, the program considers the environment dark and turns ON the lighting LED.

When the value is 500 or above, the LED is turned OFF.

Step 5: Motion Detection

The PIR sensor is checked using `digitalRead()`.

If motion is detected:

- Motion LED turns ON.
- LCD displays "ALERT!!!"
- LCD displays "Motion Detected".
- Buzzer produces an 800 Hz tone for approximately 500 ms.

If no motion is detected:

- Motion LED turns OFF.
- LCD displays "System Normal".
- LCD displays "No Motion".

Step 6: Temperature Display

At the end of each loop, the LCD displays the measured temperature and fan status.

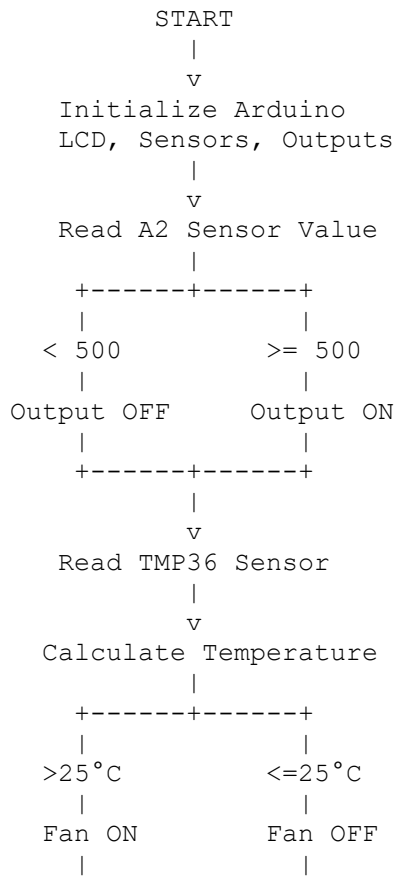
Example:

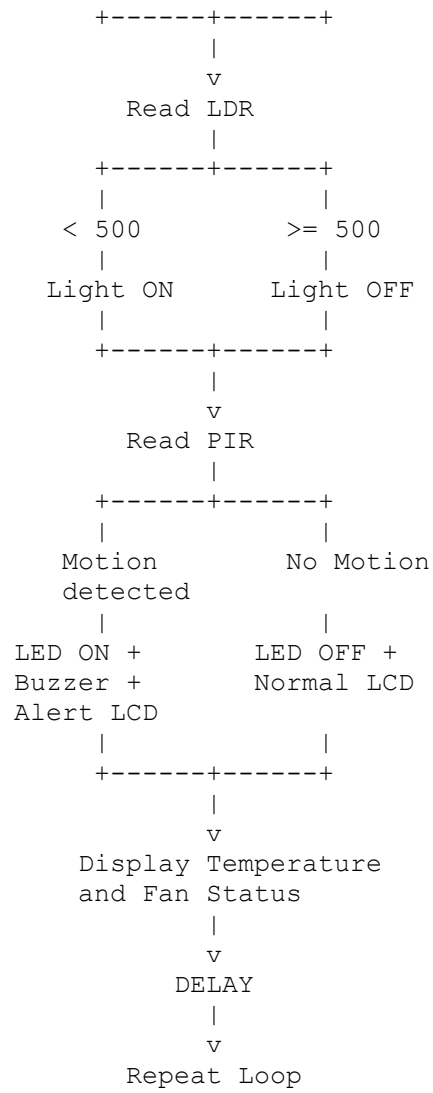
```
Temp: 27.5 C
Fan: ON
```

or:

```
Temp: 23.2 C
Fan: OFF
```

8. PROGRAM FLOWCHART





9. ALGORITHM

1. Start the Arduino.
2. Configure all input and output pins.
3. Initialize Serial communication.
4. Initialize the 16×2 LCD.
5. Display the startup message.
6. Read the additional sensor connected to A2.
7. Compare the A2 reading with 500 and control the extra output.
8. Read the TMP36 analog value.
9. Convert the analog value into voltage.
10. Calculate temperature in Celsius.
11. Compare temperature with 25°C.
12. Turn the fan ON if temperature is above 25°C.
13. Otherwise turn the fan OFF.
14. Read the LDR value from A0.
15. Turn the light LED ON when the LDR reading is below 500.
16. Turn the light LED OFF otherwise.
17. Read the PIR sensor.

18. If motion is detected, activate the motion LED and buzzer.
19. Display the motion alert on the LCD.
20. If motion is not detected, turn OFF the motion LED.
21. Display normal status on the LCD.
22. Display temperature and fan status.
23. Repeat the process continuously.

10. EXPECTED OUTPUT

Startup

```
Smart System
Starting...
```

After approximately two seconds, the LCD begins displaying system information.

No Motion

```
System Normal
No Motion
```

Motion Detected

```
ALERT!!!
Motion Detected
```

The motion LED turns ON and the buzzer sounds.

Temperature Below 25°C

```
Temp: 23.5 C
Fan: OFF
```

Temperature Above 25°C

```
Temp: 28.0 C
Fan: ON
```

Dark Environment

The LDR reading becomes lower than the programmed threshold and the light LED turns ON.

Bright Environment

The light LED turns OFF.

11. SERIAL MONITOR OUTPUT

The program uses the Serial Monitor to provide sensor information.

Typical output may look like:

```
A2: HIGH
Temperature: 27.4 C | Fan: ON
LDR: 350
Dark -> Light ON
Motion detected!
```

Another possible output is:

```
A2: LOW
Temperature: 23.1 C | Fan: OFF
LDR: 750
Bright -> Light OFF
Motion not detected
```

The actual LDR, A2, and temperature values depend on the sensors and surrounding conditions.

12. TESTING

Test	Condition	Expected Result
PIR Test	Person moves in front of sensor	Motion LED and buzzer ON
PIR Test	No movement	Motion LED OFF
LDR Test	Dark environment	Light LED ON
LDR Test	Bright environment	Light LED OFF
Temperature Test	Temperature >25°C	Fan ON
Temperature Test	Temperature ≤25°C	Fan OFF
A2 Test	A2 <500	Extra output OFF
A2 Test	A2 ≥500	Extra output ON
LCD Test	System powered	LCD displays information
Serial Test	Arduino running	Sensor values printed

13. ADVANTAGES

- Simple and inexpensive implementation.
- Combines security and automation in one system.
- Automatic control reduces manual operation.
- Temperature-based fan control can improve comfort.

- Automatic lighting can help reduce unnecessary energy usage.
- PIR-based motion detection provides a basic security feature.
- LCD provides easy-to-read system information.
- Arduino makes the system easy to modify and expand.
- Additional sensors can be added to the platform.

14. LIMITATIONS

The current prototype has several limitations:

1. The fan is controlled only ON/OFF rather than according to multiple temperature levels.
2. The LDR threshold of 500 may need calibration depending on the environment.
3. The PIR sensor may require a stabilization period after startup.
4. The security system does not include remote notification.
5. The LCD only displays a limited amount of information.
6. The additional sensor connected to A2 is not identified in the program.
7. The motor requires an appropriate driver circuit for safe operation.
8. There is a pin conflict between the buzzer and extra output.

15. IMPORTANT HARDWARE SAFETY/CORRECTION

The motor/fan should **not be connected directly to Arduino pin 6**.

A suitable circuit should use:

Arduino D6 → transistor/MOSFET → motor/fan

with:

- External motor power supply.
- Common ground between Arduino and motor supply.
- Flyback diode for a conventional DC motor.
- Appropriate resistor/gate resistor as required by the switching device.

This protects the Arduino from excessive current and voltage generated by the motor.

The buzzer and extra output should also be assigned to separate Arduino pins.

16. FUTURE SCOPE

The project can be expanded into a more advanced smart-home system by adding:

- Wi-Fi or Bluetooth connectivity.
- Mobile application control.
- IoT cloud monitoring.
- Remote security notifications.
- Automatic door control.
- Gas leakage detection.
- Smoke detection.

- Humidity monitoring.
- Automatic curtain control.
- Servo-based door locking.
- Multiple temperature thresholds.
- Variable-speed fan control.
- Real-time clock and event logging.
- Camera-based security monitoring.
- Voice-control functionality.

17. CONCLUSION

The Arduino-based Smart Home Automation and Security System successfully demonstrates the integration of multiple sensors and output devices into a single embedded system.

The LDR provides automatic lighting control, while the TMP36 sensor enables temperature monitoring and automatic fan operation. The PIR sensor adds a basic security function by detecting motion and activating an LED and buzzer. The 16×2 LCD provides a simple user interface, while the Serial Monitor allows sensor values and system conditions to be observed during operation.

The project provides practical experience with Arduino programming, analog-to-digital conversion, digital input/output, PWM, sensor interfacing, LCD interfacing, and basic automation.

With suitable hardware corrections—particularly separating the buzzer and extra output pins and using a proper motor driver—the system can serve as a foundation for a more advanced IoT-based smart-home application.

18. REFERENCES

1. Arduino Uno technical documentation.
2. Arduino IDE documentation.
3. Arduino `LiquidCrystal` library documentation.
4. TMP36 temperature sensor datasheet.
5. PIR motion sensor module documentation.
6. LDR/light sensor principles and applications.
7. Basic Arduino sensor-interfacing concepts.

19. PROJECT SUMMARY

Project Title: Smart Home Automation and Security System Using Arduino

Controller: Arduino Uno

Sensors: PIR, LDR, TMP36, Additional Analog Sensor

Outputs: LCD, LEDs, Buzzer, DC Motor/Fan

Temperature Threshold: 25°C

LDR Threshold: 500

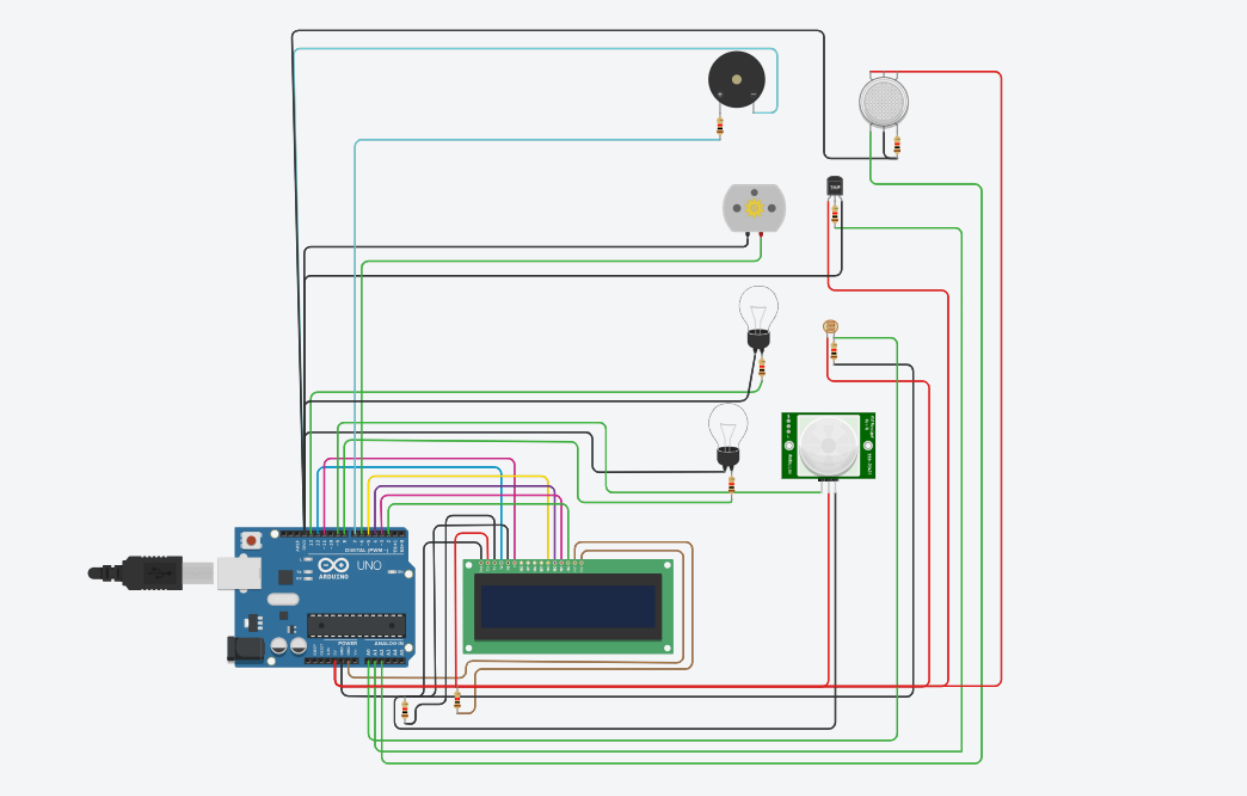
A2 Threshold: 500

Serial Communication: 9600 baud

LCD: 16×2

Main Applications: Home automation, environmental monitoring, automatic lighting, temperature control, and basic security.

20.CIRCUIT DIAGRAM:-



19. SOURCE CODE

```
#include <LiquidCrystal.h>


// ---------------- LCD ----------------

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);


// ---------------- PIN DEFINITIONS ----------------
```

```
const int PIR_PIN = 9;

const int MOTION_LED = 8;

const int BUZZER_PIN = 7;


const int LIGHT_LED = 13;

const int LDR_PIN = A0;


const int TEMP_PIN = A1;


const int EXTRA_SENSOR = A2;

const int EXTRA_OUTPUT = 10;


const int MOTOR_PIN = 6;


// ---------------- VARIABLES ----------------

int pirSensor;

int ldrValue;

int extraValue;

int tempAnalog;


float temperatureC;


// ---------------- SETUP ----------------

void setup()

{
```

```
// PIR
pinMode(PIR_PIN, INPUT);
pinMode(MOTION_LED, OUTPUT);

// Buzzer
pinMode(BUZZER_PIN, OUTPUT);

// Light
pinMode(LIGHT_LED, OUTPUT);
pinMode(LDR_PIN, INPUT);

// Temperature
pinMode(TEMP_PIN, INPUT);

// Motor
pinMode(MOTOR_PIN, OUTPUT);

// Extra sensor/output
pinMode(EXTRA_SENSOR, INPUT);
pinMode(EXTRA_OUTPUT, OUTPUT);

// Serial
Serial.begin(9600);

// LCD
```

```
  lcd.begin(16, 2);
  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("Smart System");
  lcd.setCursor(0, 1);
  lcd.print("Starting...");

  delay(2000);
  lcd.clear();
}

// ---------------- MAIN LOOP ----------------
void loop()
{
  // ================================================
  // 1. EXTRA A2 SENSOR
  // ================================================

  extraValue = analogRead(EXTRA_SENSOR);

  if (extraValue < 500)
  {
    digitalWrite(EXTRA_OUTPUT, LOW);
```

```
  Serial.println("A2: LOW");
}
else
{
  digitalWrite(EXTRA_OUTPUT, HIGH);

  Serial.println("A2: HIGH");
}



// ================================================
// 2. TEMPERATURE SENSOR + FAN
// ================================================

tempAnalog = analogRead(TEMP_PIN);

// For TMP36 temperature sensor
float voltage = tempAnalog * (5.0 / 1023.0);

temperatureC = (voltage - 0.5) * 100.0;

Serial.print("Temperature: ");
Serial.print(temperatureC);
Serial.print(" C");
```

```
if (temperatureC > 25.0)
{
  // PWM range is 0-255
  analogWrite(MOTOR_PIN, 255);

  Serial.println(" | Fan: ON");
}
else
{
  analogWrite(MOTOR_PIN, 0);

  Serial.println(" | Fan: OFF");
}



// ================================================
// 3. LDR AUTOMATIC LIGHT
// ================================================

ldrValue = analogRead(LDR_PIN);

Serial.print("LDR: ");
Serial.println(ldrValue);
```

```
if (ldrValue < 500)
{
  // Dark
  digitalWrite(LIGHT_LED, HIGH);

  Serial.println("Dark -> Light ON");
}
else
{
  // Bright
  digitalWrite(LIGHT_LED, LOW);

  Serial.println("Bright -> Light OFF");
}



// ================================================
// 4. PIR MOTION SENSOR
// ================================================

pirSensor = digitalRead(PIR_PIN);

if (pirSensor == HIGH)
{
  // Motion detected
```

```
  digitalWrite(MOTION_LED, HIGH);

  Serial.println("Motion detected!");

  // LCD
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("ALERT!!!");

  lcd.setCursor(0, 1);
  lcd.print("Motion Detected");

  // Buzzer
  tone(BUZZER_PIN, 800, 500);
 delay(500);
}
else
{
  // No motion
  digitalWrite(MOTION_LED, LOW);

  Serial.println("Motion not detected");

  lcd.clear();
  lcd.setCursor(0, 0);
```

```
  lcd.print("System Normal");


  lcd.setCursor(0, 1);
  lcd.print("No Motion");


  delay(500);
}



// ================================================
// 5. DISPLAY TEMPERATURE
// ================================================
 lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Temp: ");
lcd.print(temperatureC, 1);
lcd.print(" C");
lcd.setCursor(0, 1);
if (temperatureC > 25.0)
{
  lcd.print("Fan: ON ");
}
else
{
  lcd.print("Fan: OFF");
```

```
  }
  delay(1000);
}
```

Thank You